Programarea calculatoarelor

FMI

Secția Calculatoare și tehnologia informației, anul I

Cursul 9 / 08.12.2025

Programa cursului

1. Introducere
- Algoritmi
- Limbaje de programare.
- Introducere în limbajul C. Structura unui program C

2. Fundamentele limbajului C
- Etapele realizării unui program C
- Tipuri de date fundamentale. Variabile. Constante. Operatori. Expresii. Conversii
- Tipuri derivate de date: tablouri, șiruri de caractere, structuri, uniuni, câmpuri de biți, enumerări, pointeri
- Instrucțiuni de control
- Directive de preprocesare. Macrodefiniții
- Funcții de citire/scriere

3. Fișiere text
- Funcții specifice de manipulare

4. Funcții (1)
- Declarare și definire. Apel.
- Metode de trasmitere a paramerilor.
- Pointeri la funcții

5. Tablouri și pointeri
- Legătura dintre tablouri și pointeri
- Aritmetica pointerilor
- Alocarea dinamică a memoriei
- Clase de memorare

6. Șiruri de caractere
- Funcții specifice de manipulare

7. Fișiere binare
- **Funcții specifice de manipulare**

8. Structuri de date complexe și autoreferite
- Definire și utilizare

9. Funcții (2)
- Funcții cu număr variabil de argumente
- Preluarea argumentelor funcției main din linia de comandă
- Programare generică

Cuprinsul cursului de azi

1. **Recapitulare. Lucrul cu fișiere**
2. **Fișiere binare**
3. **Deschiderea. Funcții de citire/ scriere**
4. **Alte funcții**

Recapitulare. Lucrul cu fișiere

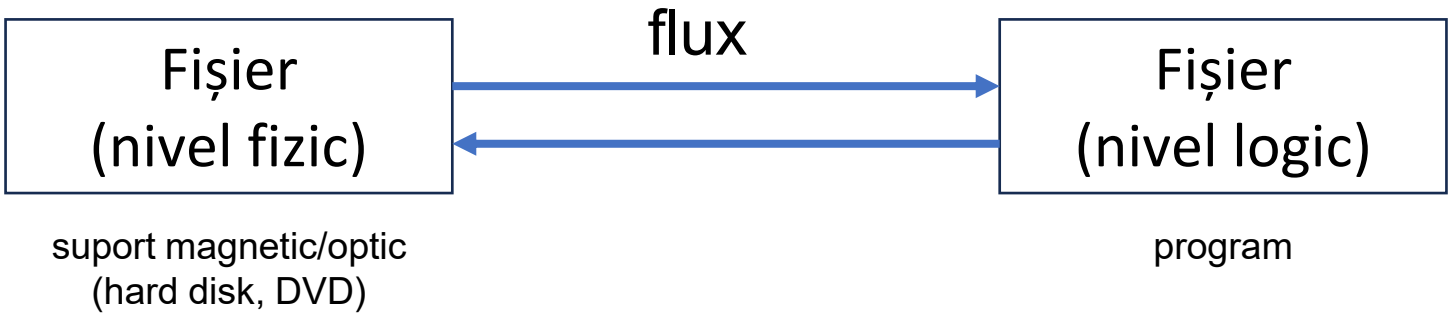
- **Fișier = o colecție de date (de obicei de același tip), stocate pe suport extern.**
- Fișierele sunt entități ale sistemului de operare: numele lor respectă convențiile sistemului, fără legătură cu un anumit limbaj de programare.
- Conținutul unui fișier este variabil (numărul de elemente poate fi chiar nul) și poate fi format din: **texte, numere, informații binare** (executabile, numere, imagini sau sunete în format binar).
- Fișierele se folosesc în principal pentru păstrarea permanentă a unor date importante, dar și pentru memorarea unor date inițiale (de intrare) sau date de ieșire numerice voluminoase, de interes pentru anumite programe.
- Noțiunea de fișier include orice flux de date (**I/O stream**) din exterior spre memorie și viceversa.

Recapitulare. Lucrul cu fișiere

- programe sursă: .c, .cpp
- executabile
- imagini: .jpeg, .jpg, .png, .bmp
- documente: .pdf, .dvi, .eps
- audio: .mp3
- video: .avi, .mp4

- pentru fiecare tip de fişier binar este necesar un program care să cunoască şi să interpreteze corect datele din fişier

Recapitulare. Lucrul cu fișiere



- ne referim la un fișier fizic într-un program C printr-o variabilă
- asocierea dintre numele extern al un fișier (un șir de caractere) și o variabilă dintr-un program C se realizează la deschiderea unui fișier, printr-o funcție standard

Recapitulare. Lucrul cu fișiere

Operațiile pentru prelucrarea fișierelor sunt:

- crearea fișierului;
- asignarea fișierului intern (logic) la unul extern (fizic);
- deschiderea fișierului;
- operații de acces la date;
- închiderea fișierului.

Recapitulare. Lucrul cu fișiere

Există fișiere standard, care sunt gestionate automat de sistem, dar asupra cărora se poate interveni și în mod explicit:

- fișierul standard de intrare **stdin** (STanDard INput)
- fișierul standard de ieşire **stdout** (STanDard OUTput)
- fișierul standard scriere mesaje de eroare **stderr** (STanDard ERRor)

Recapitulare. Lucrul cu fișiere

Tipuri de fișiere:

- **fișiere text: octeții (caractere ASCII) sunt organizați pe linii.**

✓ **Caracterele terminatorii de linii sunt:**

- Unix: caracterul line feed (LF) = '\n' – cod ASCII 10
- Mac OS vechi : = '\r' – cod ASCII 13
- Windows: CR + LF, caracterul cariage return (CR) - cod ASCII 13
- un fișier text poate fi terminat printr-un caracter terminator de fișier (EOF = CTRL-Z). Acest terminator nu este obligatoriu. Sfârşitul unui fişier disc poate fi detectat şi pe baza lungimii fişierului (număr de octeţi), memorată pe disc.

- **fișiere binare: octeții nu sunt organizați în nici un fel (nu există noțiunea de linie)**

Categorii de funcții folosite pentru prelucrarea fișierelor

- funcții de prelucrare generală: se aplică tuturor fișierelor, indiferent de tipul informației conținute
- funcții de citire/scriere **cu format**: pentru citirea/scrierea **fișierelor text** care conțin informație de tip text (linii de text, separate prin perechea CR/LF, iar la sfârșit se găsește caracterul CTRL-Z)
- funcții de citire/scriere **fără format**: pentru citirea/scrierea **fișierelor binare**
- funcții de citire/scriere pentru **caractere**: pentru transferul unui singur caracter între un **fișier text** și memorie
- funcții de citire/scriere pentru **șiruri de caractere**: pentru transferul unui șir de caractere între un **fișier text** și memorie

Funcțiile de citire/scriere deplasează pointerul de citire/scriere al fișierului spre sfârșitul acestuia, cu un număr de octeți egal cu numărul de octeți transferați fără a trece de sfârșitul fișierului.

Recapitulare. Lucrul cu fișiere

- în biblioteca **stdio.h** este definită o structură **FILE**. Această structură conține (în membri ei) informații referitoare la un fișier:
  - ✓nume,
  - ✓adresa,
  - ✓adresa bufferului intern în care se procesează (citire/scriere) octeții din fișier,
  - ✓indicator de sfârșit de fișier,
  - ✓indicator de poziție în fișier, etc.
- lucrul cu fișiere presupune declararea unui pointer la structura **FILE** în vederea realizării legăturii dintre nivelul logic (variabila fișier) și nivelul fizic (numele extern al fișierului) :

FILE * f;

Recapitulare. Lucrul cu fișiere

Deschiderea unui fişier existent, precum şi crearea unui fişier nou se fac cu ajutorul funcţiei **fopen**, care are următorul prototip:

FILE *fopen (const char *cale_nume, const char *mod);

unde:

- cale_nume este un pointer spre un şir de caractere care defineşte calea de nume a fişierului
- mod este un pointer spre un şir de caractere care defineşte modul de prelucrare a fişierului deschis

Recapitulare. Lucrul cu fișiere

Cum se defineşte modul de prelucrare a unui fişier deschis:

“r” – citire în mod text (read);

“w” – scriere în mod text (write);

“a” – adăugare în mod text (append);

“r+” – citire/scriere în mod text (modificare);

“w+” – citire/scriere în mod text (creare);

“rb” – citire în mod binar;

“wb” – scriere în mod binar;

“r+b” – citire/scriere în mod binar (modificare);

“w+b” – citire/scriere în mod binar (creare);

“ab” – adăugare de înregistrări în mod binar

Recapitulare. Lucrul cu fișiere

- **detectarea sfârșitului de fișier:** se poate realiza și folosind funcția **feof** (find end of file) :

int feof(FILE *f)

unde f = pointer la fișierul pe care îl prelucrez

- funcția **feof** returnează:
  - ✓ 0 dacă nu s-a ajuns la sfârșitul fișierului la ultima operație de citire
  - ✓ o valoare nenulă dacă s-a ajuns la sfârșitul fișierului

```
while (!feof(f))
        citeste x din f;
        scrie x
```

(Scrie un x = -1 în plus)

```
while (1)
        citeste x din f;
        if (feof(f)) break;
        scrie x
```

- Detectarea sfârșitului de fișier

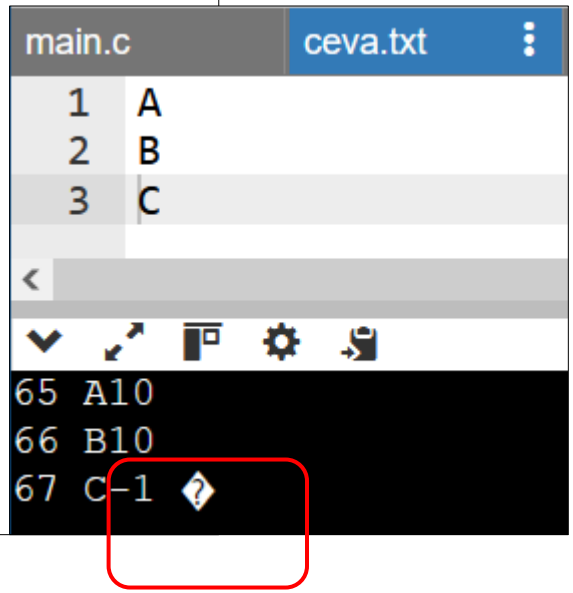
```c
#include <stdio.h>

void main()
{
    FILE *f;
    int c;

    f = fopen("ceva.txt", "r");
    if (f == NULL) {
        printf("Eroare la deschiderea fisierului!\n");
        return; // Ies din program.
    }

    while(!feof(f))
    {
        c = fgetc(f);
        printf("%d %c", c, c);
    }

    fclose(f);
}
```



- Detectarea sfârșitului de fișier

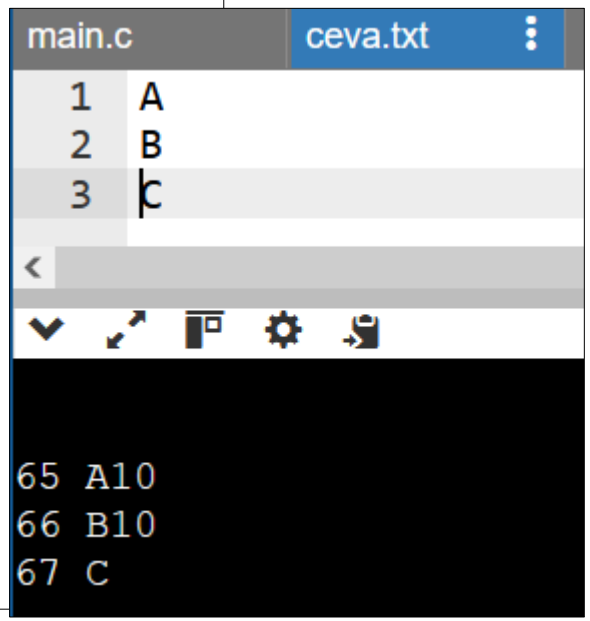
```c
#include <stdio.h>

void main()
{
    FILE *f;
    int c;

    f = fopen("ceva.txt", "r");
    if (f == NULL)
    {
        printf("Eroare la deschiderea fisierului!\n");
        return;
    }

    while(1)
    {
        c = fgetc(f);
        if(feof(f))
            break;
        printf("%d %c", c, c);
    }

    fclose(f);
}
```



- Aflarea numărului de linii al unui fișier

```c
#include <stdio.h>

void main()
{
    FILE *f;
    char linie[1000];
    int numarLinii = 0;

    f = fopen("ceva.txt", "r");
    if (f == NULL) {
        printf("Eroare la deschiderea fisierului!\n");
        return; // Ies din program.
    }

    while(fgets(linie, 1000, f) != NULL)
        numarLinii++;

    printf("Fisierul contine %d linii.\n", numarLinii);

    fclose(f);
}
```

Cuprinsul cursului de azi

1. Recapitulare. Lucrul cu fișiere
2. **Fișiere binare**
3. Deschiderea. Funcții de citire/ scriere
4. Alte funcții

Fișiere binare

- **fișier binar = șir de octeți neformatat pe linii care este stocat pe suport magnetic/optic. Octeții nu sunt considerați ca fiind coduri de caractere.**
- un fişier binar este format în general din **articole** de lungime fixă, fără separatori între articole
- un articol poate conţine:
  - ✓ un singur octet
  - ✓ un număr binar (pe 2, 4 sau 8 octeţi)
  - ✓ o structură cu date de diferite tipuri



Fișier binar vs fișier text

- **fișier text** = șir de octeți organizat pe linii, stocat pe suport magnetic/optic, în care fiecare octet este interpretat drept caracter cu codul ASCII dat de octetul respectiv.
- **fișier binar** = șir de octeți neformatat pe linii care este stocat pe suport magnetic/optic. Octeții nu sunt considerați ca fiind coduri de caractere.

- Diferența fișier binar – fișier text:
  - ✓ un **fişier binar** se accesează ca **o succesiune de octeţi**, cărora funcţiile de citire şi scriere din fişier nu le dau nici o interpretare
  - ✓ un **fişier text** se accesează ca **o succesiune de linii de text** de lungime variabilă (încheiate cu un terminator de linie : '\n') utilizând un set dedicat de funcţii din biblioteca standard

Cuprinsul cursului de azi

1. Recapitulare. Lucrul cu fișiere
2. Fișiere binare
3. **Deschiderea. Funcții de citire/ scriere**
4. Alte funcții

Deschiderea fișierelor

FILE *fopen(char *nume_fisier, char *mod_deschidere)

- nume_fisier = numele fisierului
- mod_deschidere = șir de caractere ce precizează tipul de acces la fișier:

Mod	Semnificaţie
r	Deschide un fişier tip text pentru a fi citit
w	Creează un fişier tip text pentru a fi scris
a	Adaugă într-un fişier tip text
rb	Deschide un fişier de tip binar pentru a fi citit
wb	Creează un fişier de tip binar pentru a fi scris
ab	Adaugă într-un fişier de tip binar
r+	Deschide un fişier tip text pentru a fi citit/scris
w+	Creează un fişier tip text pentru a fi citit/scris
a+	Adaugă în sau creează un fişier tip text pentru a fi citit/scris
r+b	Deschide un text în binar pentru a fi citit/scris
w+b	Creează un fişier de tip binar pentru a fi citit/scris
a+b	Adaugă sau creează un fişier de tip binar pentru a fi citit/scris

Crearea/ închiderea unui fişier binar

- se definește un pointer la tipul FILE,
- se apelează funcția **fopen** cu parametru mod “**wb**”,
- se atribuie valoarea returnată pointerului la tipul FILE

- Dacă valoarea returnată este pointerul NUL, operația de creare a eșuat.

Crearea/ închiderea unui fişier binar

```c
#include <stdio.h>

int main()
{
    FILE *f; // pointer la tipul FILE
    char fName[30]; // identificatorul (numele extern) fisierului
    printf("Nume fisier: ");
    scanf("%s", fName);
    if ((f = fopen(fName, "wb")) ==  NULL)
        printf("Fisierul %s nu a fost creat.", fName);
    else
    {
        printf("Fisierul %s a fost creat.", fName);
        fclose(f);
    }
    return 0;
}
```

Deschiderea unui fişier binar

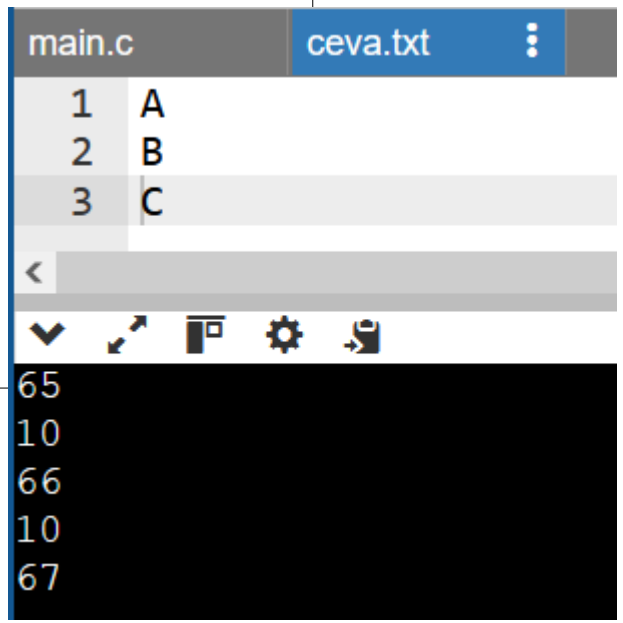
```c
#include <stdio.h>

void main()
{
    FILE *f;
    int c;

    f = fopen("ceva.txt", "rb");
    if (f == NULL)
    {
        printf("Eroare la deschiderea fisierului!\n");
        return;
    }

    while((c = fgetc(f)) != EOF)
        printf("%d\n", c);

    fclose(f);
}
```



Funcții de citire/ scriere

int fwrite(void *tablou, int dimElem, int nrElem, FILE *f)

- **scrie** în fișierul referit de **f** cel mult **nrElem** elemente de dimensiune **dimElem** de la adresa **tablou**;

int fread(void *tablou, int dimElem, int nrElem, FILE *f)

- **citește** cel mult **nrElem** elemente de dimensiune **dimElem** din fisierul referit de **f** la adresa **tablou**.

Funcții de citire/ scriere

Ex.1. Scriere

```c
#include <stdio.h>

int main()
{
    FILE *f, *g;
    int x = 1936880995;

    f = fopen("f_nrb.out", "wb");
    g = fopen("f_nrt.out", "w");

    fwrite(&x, sizeof(int), 1, f);
    fprintf(g, "%d", x);

    printf("Cod ASCII c = %d\n", 'c');
    printf("Cod ASCII u = %d\n", 'u');
    printf("Cod ASCII r = %d\n", 'r');
    printf("Cod ASCII s = %d\n", 's');

    printf("%d\n", (115<<24) + (114<<16) + (117<<8) + 99);

    fclose(f); fclose(g);

    return 0;
}
```

Funcții de citire/ scriere

```c
#include <stdio.h>

int main()
{
    FILE *f, *g;
    int x = 1936880995;

    f = fopen("f_nrb.out", "wb");
    g = fopen("f_nrt.out", "w");

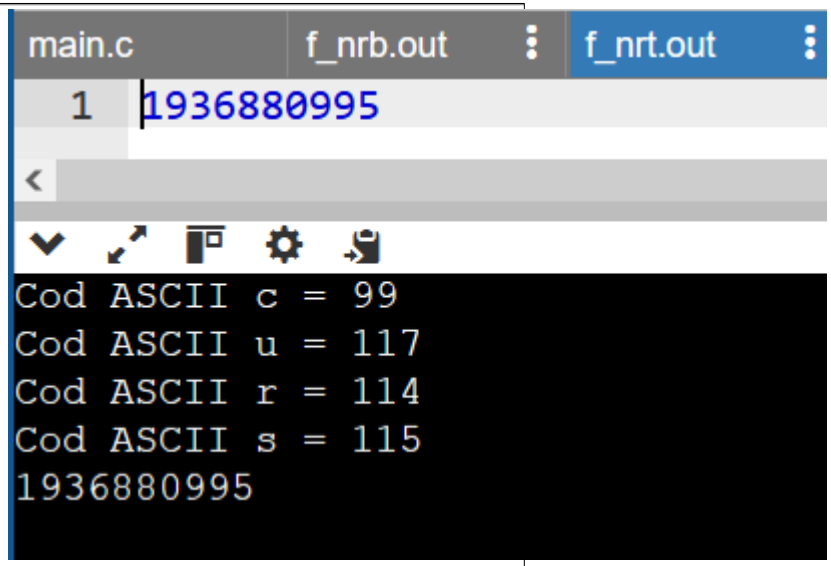
    fwrite(&x, sizeof(int), 1, f);
    fprintf(g, "%d", x);

    printf("Cod ASCII c = %d\n", 'c');
    printf("Cod ASCII u = %d\n", 'u');
    printf("Cod ASCII r = %d\n", 'r');
    printf("Cod ASCII s = %d\n", 's');

    printf("%d\n", (115<<24) + (114<<16) + (117<<8) + 99);

    fclose(f); fclose(g);

    return 0;
}
```



Funcții de citire/ scriere

```c
#include <stdio.h>

int main()
{
    FILE *f, *g;
    int x = 1936880995;

    f = fopen("f_nrb.out", "wb");
    g = fopen("f_nrt.out", "w");

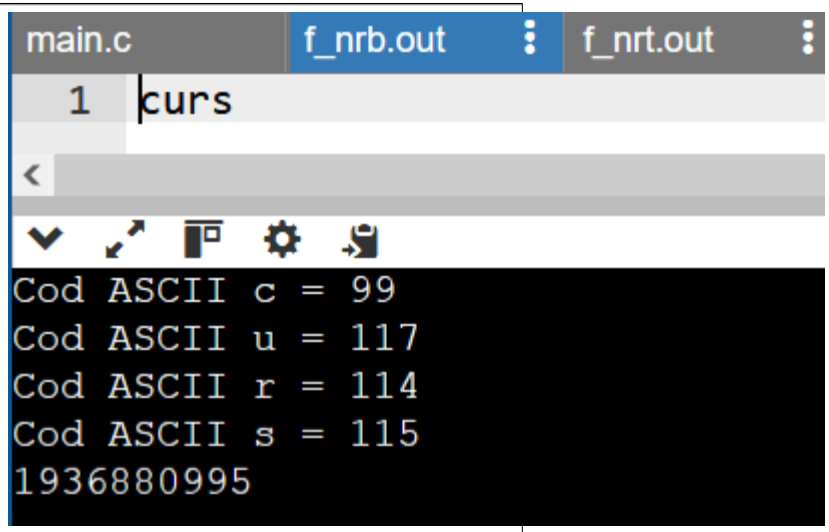
    fwrite(&x, sizeof(int), 1, f);
    fprintf(g, "%d", x);

    printf("Cod ASCII c = %d\n", 'c');
    printf("Cod ASCII u = %d\n", 'u');
    printf("Cod ASCII r = %d\n", 'r');
    printf("Cod ASCII s = %d\n", 's');

    printf("%d\n", (115<<24) + (114<<16) + (117<<8) + 99);

    fclose(f); fclose(g);

    return 0;
}
```



Funcții de citire/ scriere

```c
#include <stdio.h>

int main()
{
    FILE *f, *g;
    int x = 1936880995;

    f = fopen("f_nrb.out", "wb");
    g = fopen("f_nrt.out", "w");

    fwrite(&x, sizeof(int), 1, f);
    fprintf(g, "%d", x);

    printf("Cod ASCII c = %d\n", 'c');
    printf("Cod ASCII u = %d\n", 'u');
    printf("Cod ASCII r = %d\n", 'r');
    printf("Cod ASCII s = %d\n", 's');

    printf("%d\n", (115<<24) + (114<<16) + (117<<8) + 99);

    fclose(f); fclose(g);

    return 0;
}
```

Scrierea numărului 1936880995 în baza 2:

octetul 3 octetul 2 octetul 1 octetul 0

01110011 01110010 01110101 01100011

s = 115 r = 114 u = 117 c = 99

Funcții de citire/ scriere

Ex.2. Citire

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *f;
    int x; char c;

    f = fopen("f_nrb.out", "rb");
    if (f == NULL)
    {
        printf("Deschidere esuata.");
        exit(1);// din stdlib.h
    }

    while(fread(&x, sizeof(int), 1, f) == 1)
        printf("x = %d\n", x);

    fclose(f);

    f = fopen("f_nrb.out", "rb");

    while(fread(&c, sizeof(char), 1, f) == 1)
        printf("%c = %d\n", c, c);
    fclose(f);

    return 0;
}
```

Funcții de citire/ scriere

Ex.2. Citire

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *f;
    int x; char c;

    f = fopen("f_nrb.out", "rb");
    if (f == NULL)
    {
        printf("Deschidere esuata.");
        exit(1);// din stdlib.h
    }

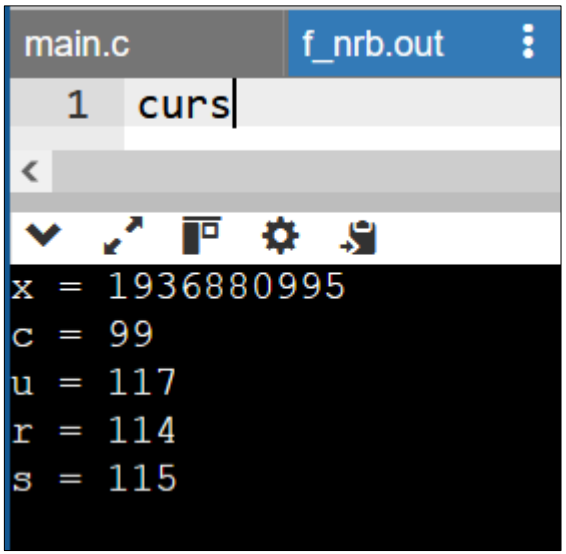
    while(fread(&x, sizeof(int), 1, f) == 1)
        printf("x = %d\n", x);

    fclose(f);

    f = fopen("f_nrb.out", "rb");

    while(fread(&c, sizeof(char), 1, f) == 1)
        printf("%c = %d\n", c, c);
    fclose(f);

    return 0;
}
```



Funcții de citire/ scriere

Ex.3. Scriere

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *f1, *f2, *f3;

    f1 = fopen("fb1.txt", "wb");
    f2 = fopen("fb2.txt", "wb");
    f3 = fopen("fb3.txt", "wb");

    if ((f1 == NULL) || (f1 == NULL) || (f1 == NULL))
    {   printf("Deschidere esuata."); exit(1); }

    int v[] = {50, 51, 52, 53}, i;

    for(i = 0; i < 4; i++)
        fwrite(&v[i], sizeof(int), 1, f1);

    fwrite(v, sizeof(int), 4, f2);

    fwrite( v, 4*sizeof(int), 1, f3);

    fclose(f1); fclose(f2); fclose(f3);

    return 0;
}
```

Funcții de citire/ scriere

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *f1, *f2, *f3;

    f1 = fopen("fb1.txt", "wb");
    f2 = fopen("fb2.txt", "wb");
    f3 = fopen("fb3.txt", "wb");

    if ((f1 == NULL) || (f1 == NULL) || (f1 == NULL))
    {   printf("Deschidere esuata."); exit(1); }

    int v[] = {50, 51, 52, 53}, i;

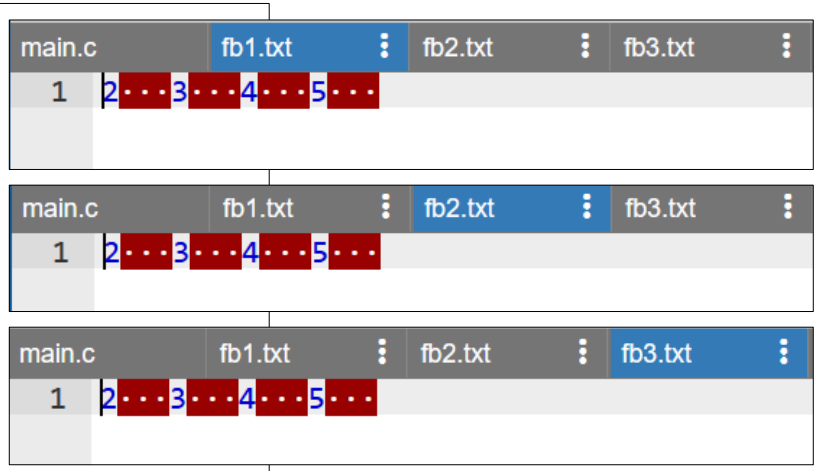
    for(i = 0; i < 4; i++)
        fwrite(&v[i], sizeof(int), 1, f1);

    fwrite(v, sizeof(int), 4, f2);

    fwrite( v, 4*sizeof(int), 1, f3);

    fclose(f1); fclose(f2); fclose(f3);

    return 0;
}
```



Funcții de citire/ scriere

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *f1, *f2, *f3;

    f1 = fopen("fb1.txt", "wb");
    f2 = fopen("fb2.txt", "wb");
    f3 = fopen("fb3.txt", "wb");

    if ((f1 == NULL) || (f1 == NULL) || (f1 == NULL))
    {   printf("Deschidere esuata."); exit(1); }

    int v[] = {50, 51, 52, 53}, i;

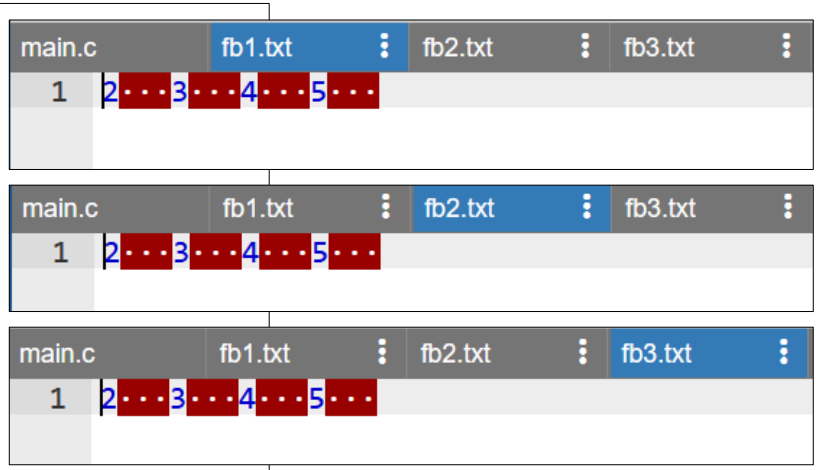
    for(i = 0; i < 4; i++)
        fwrite(&v[i], sizeof(int), 1, f1);

    fwrite(v, sizeof(int), 4, f2);

    fwrite( v, 4*sizeof(int), 1, f3);

    fclose(f1); fclose(f2); fclose(f3);

    return 0;
}
```



Dec	Hx	Oct	Html	Chr
32	20	040		Space
33	21	041	!	!
34	22	042	"	"
35	23	043	#	#
36	24	044	$	$
37	25	045	%	%
38	26	046	&	&
39	27	047	'	'
40	28	050	(	(
41	29	051	)	)
42	2A	052	*	*
43	2B	053	+	+
44	2C	054	,	,
45	2D	055	-	-
46	2E	056	.	.
47	2F	057	/	/
48	30	060	0	0
49	31	061	1	1
50	32	062	2	2
51	33	063	3	3
52	34	064	4	4
53	35	065	5	5

Funcții de citire/ scriere

Ex.4. Citire

```c
#include <stdio.h>
#include <stdlib.h>

void afisare(int*, int);

int main()
{
    FILE *f1, *f2, *f3;

    f1 = fopen("fb1.txt", "rb");
    f2 = fopen("fb2.txt", "rb");
    f3 = fopen("fb3.txt", "rb");

    if ((f1 == NULL) || (f1 == NULL) || (f1 == NULL))
    {   printf("Deschidere esuata."); exit(1); }

    int v[5], i;

    for(i = 0; i < 4; i++) fread(&v[i], sizeof(int), 1, f1);
    afisare(v, 4);

    fread(v, sizeof(int), 4, f2); afisare(v, 4);

    fread( v, 4*sizeof(int), 1, f3); afisare(v, 4);

    fclose(f1); fclose(f2); fclose(f3);

    return 0;
}
```

Funcții de citire/ scriere

Ex.4. Citire

```c
#include <stdio.h>
#include <stdlib.h>

void afisare(int*, int);

int main()
{
    FILE *f1, *f2, *f3;

    f1 = fopen("fb1.txt", "rb");
    f2 = fopen("fb2.txt", "rb");
    f3 = fopen("fb3.txt", "rb");

    if ((f1 == NULL) || (f1 == NULL) || (f1 == NULL))
    {   printf("Deschidere esuata."); exit(1); }

    int v[5], i;

    for(i = 0; i < 4; i++) fread(&v[i], sizeof(int), 1, f1);
    afisare(v, 4);

    fread(v, sizeof(int), 4, f2); afisare(v, 4);

    fread( v, 4*sizeof(int), 1, f3); afisare(v, 4);

    fclose(f1); fclose(f2); fclose(f3);

    return 0;
}
```

```c
void afisare(int* v, int dim)
{
    for(int i = 0; i < dim; i++)
        printf("%d ", v[i]);
    printf("\n");
}
```

```
50 51 52 53
50 51 52 53
50 51 52 53
```

Funcții de citire/ scriere

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct{
        char nume[20];
        float medie;
    }student;

int main()
{
    FILE *f = fopen("struct.out", "wb");

    if (f == NULL)
    { printf("Deschidere esuata."); exit(1); }

    student s;
    strcpy(s.nume, "Alexandru");
    s.medie = 9.5;

    fwrite(&s, sizeof(s), 1, f);

    fclose(f);

    return 0;
}
```

Ex. 5. Scrierea unei structuri

Funcții de citire/ scriere

Ex. 5. Scrierea unei structuri

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct{
        char nume[20];
        float medie;
    }student;

int main()
{
    FILE *f = fopen("struct.out", "wb");

    if (f == NULL)
    { printf("Deschidere esuata."); exit(1); }

    student s;
    strcpy(s.nume, "Alexandru");
    s.medie = 9.5;

    fwrite(&s, sizeof(s), 1, f);

    fclose(f);

    return 0;
}
```

main.c | struct.out

```
1  Alexandru·ÂGhz··ð··Y···A
```

Funcții de citire/ scriere

Ex. 6. Citirea unei structuri

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct{
        char nume[20];
        float medie;
    }student;

int main()
{
    FILE *f = fopen("struct.out", "rb");

    if (f == NULL)
    { printf("Deschidere esuata."); exit(1); }

    student s;
    fread(&s, sizeof(s), 1, f);

    printf("%s are media %.2f. \n", s.nume, s.medie);

    fclose(f);

    return 0;
}
```

Funcții de citire/ scriere

Ex. 6. Citirea unei structuri

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct{
        char nume[20];
        float medie;
    }student;

int main()
{
    FILE *f = fopen("struct.out", "rb");

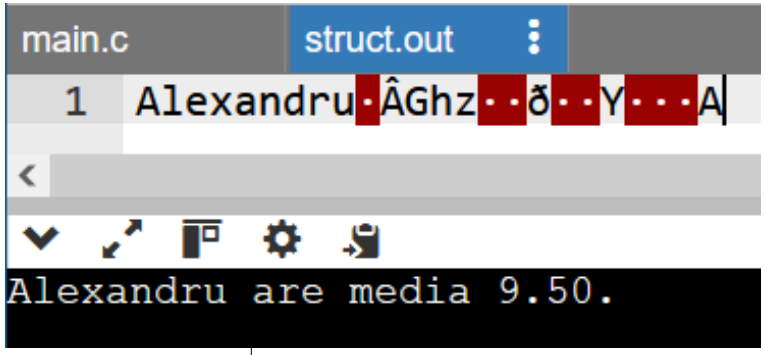
    if (f == NULL)
    { printf("Deschidere esuata."); exit(1); }

    student s;
    fread(&s, sizeof(s), 1, f);

    printf("%s are media %.2f. \n", s.nume, s.medie);

    fclose(f);

    return 0;
}
```



Funcții de citire/ scriere

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct{
        char nume[20];
        int varsta;
    }persoana;

int main()
{
    FILE *f, *g;
    f = fopen("date.out", "wb");
    if (f == NULL)
    { printf("Deschidere esuata."); exit(1); }

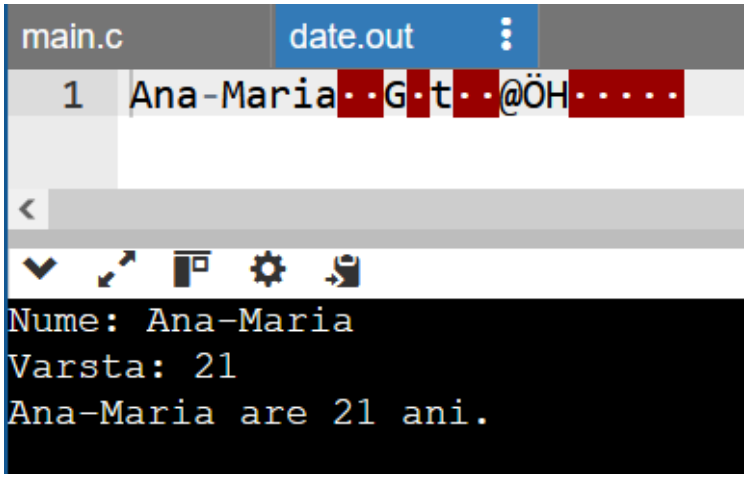
    persoana p, t;

    printf("Nume: "); scanf("%s", p.nume);
    printf("Varsta: "); scanf("%d", &p.varsta);
    fwrite(&p, sizeof(p), 1, f);
    fclose(f);

    g = fopen("date.out", "rb");
    fread(&t, sizeof(t), 1, g);
    printf("%s are %d ani.", t.nume, t.varsta);
    fclose(g);

    return 0;
}
```

Ex.8. Scrierea/ citirea unei structuri



Exemplu: Scrieți un program C care să creeze un fișier cu numele **student.txt** pentru citirea numelui și notei unui număr de studenți stabilit de utilizator.

main.c

```c
#include <stdio.h>

int main()
{
    char name[50];
    int nota,i,n;
    printf("Numarul de studenti: ");
    scanf("%d",&n);
    FILE *f;
    f=(fopen("F:\\temp\\student.txt","w"));
    if(f==NULL)
        printf("Err!");
    else
        for(i=0;i<n;++i)
        {
            printf("Pentru studentul %d\nInserati numele: ",i+1);
            scanf("%s",name);
            printf("Inserati nota: ");
            scanf("%d",&nota);
            fprintf(f,"\nNume: %s \nnota=%d \n",name,nota);
        }
    fclose(f);
return 0;
}

```

Exemplu: Scrieți un program C care sa creeze un fișier cu numele **student.txt** pentru citirea numelui si notei unui număr de studenți stabilit de utilizator.

```c
#include <stdio.h>

int main()
{
    char name[50];
    int nota,i,n;
    printf("Numarul de studenti: ");
    scanf("%d",&n);
    FILE *f;
    f=(fopen("F:\\temp\\student.txt","w"));
    if(f==NULL)
        printf("Err!");
    else
        for(i=0;i<n;++i)
        {
            printf("Pentru studentul %d\nInserati numele: ",i+1);
            scanf("%s",name);
            printf("Inserati nota: ");
            scanf("%d",&nota);
            fprintf(f,"\nNume: %s \nnota=%d \n",name,nota);
        }
    fclose(f);
return 0;
}
```

main.c | F:\temp\student.txt

```
1
2  Nume: Pop
3  nota=9
4
5  Nume: Albu
6  nota=10
7
```

```
Numarul de studenti: 2
Pentru studentul 1
Inserati numele: Pop
Inserati nota: 9
Pentru studentul 2
Inserati numele: Albu
Inserati nota: 10
```

Reluarea rulării are ca efect pierderea datelor din fișier.

Exemplu: Scrieți un program C care să creeze un fișier cu numele **student.txt** pentru citirea numelui și notei unui număr de studenți stabilit de utilizator. *Dacă fișierul există, să se adauge informațiile în fișierul existent.*

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    char nume[50];
    int nota, i, n;
    printf("Numarul de studenti: ");
    scanf("%d", &n);
    FILE *f;
    f = fopen("F:\\temp\\student.txt", "a");
    if(f == NULL)
        { printf("Eroare!"); exit(1);}
    else
        for(i = 0; i < n; i++)
        {
            printf("Studentul %d:\nnume: ", i + 1);
            scanf("%s", nume);
            printf("nota: ");
            scanf("%d", &nota);
            fprintf(f, "Nume: %s nota: %d\n", nume, nota);
        }
    fclose(f);

    return 0;
}
```

main.c F:\temp\student.txt

```
Nume: Daniel nota: 9
Nume: Bogdan nota: 10

```

```
Numarul de studenti: 2
Studentul 1:
nume: Daniel
nota: 9
Studentul 2:
nume: Bogdan
nota: 10
```

Exemplu: Scrieți un program C care sa creeze un fișier cu numele **student.txt** pentru citirea numelui si notei unui număr de studenți stabilit de utilizator. *Dacă fișierul există, să se adauge informațiile în fișierul existent.*

main.c F:\temp\student.txt

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    char nume[50];
    int nota, i, n;
    printf("Numarul de studenti: ");
    scanf("%d", &n);
    FILE *f;
    f = fopen("F:\\temp\\student.txt", "a");
    if(f == NULL)
        { printf("Eroare!"); exit(1);}
    else
        for(i = 0; i < n; i++)
        {
            printf("Studentul %d:\nnume: ", i + 1);
            scanf("%s", nume);
            printf("nota: ");
            scanf("%d", &nota);
            fprintf(f, "Nume: %s nota: %d\n", nume, nota);
        }
    fclose(f);

    return 0;
}
```

main.c F:\temp\student.txt

```
1  Nume: Daniel nota: 9
2  Nume: Bogdan nota: 10
3  Nume: Ana nota: 9
4  Nume: Florin nota: 8
5
```

```
Numarul de studenti: 2
Studentul 1:
nume: Ana
nota: 9
Studentul 2:
nume: Florin
nota: 8
```

Exemplu:

```c
struct student{
    char nume[30]; int nota;} s;

int main()
{
    char idF[50]; FILE *f; int n, i;
    printf("Identificator fisier "); scanf("%s", idF);
    f = fopen(idF, "wb"); // creare fisier binar
    if(f == NULL) { printf("Eroare!"); exit(1);}

    printf("Numar studenti: "); scanf("%d", &n);
    for(i = 1; i <= n; i++)
    {
        printf("Nume student %d: ", i); scanf("%s", s.nume);
        printf("nota studentului %s: ", s.nume); scanf("%d", &s.nota);
        fwrite(&s, sizeof(s), 1, f);
    }
    fclose(f);

    if((f = fopen(idF, "rb")) == NULL) // deschidere fisier binar
        { printf("Eroare deschidere fisier sursa!"); exit(1);}
    printf("Fisierul %s contine studentii:\n", idF);
    for(i = 1; i <= n; i++)
    {
        fread(&s, sizeof(s), 1, f); //citire din fisier
        printf("%s are nota %d \n", s.nume, s.nota);
    }
    fclose(f);
```

Exemplu:

```c
struct student{
    char nume[30]; int nota;}

int main()
{
    char idF[50]; FILE *f; in
    printf("Identificator fis
    f = fopen(idF, "wb"); //
    if(f == NULL) { printf("E

    printf("Numar studenti: "
    for(i = 1; i <= n; i++)
    {
        printf("Nume student
        printf("nota studentu
        fwrite(&s, sizeof(s),
    }
    fclose(f);

    if((f = fopen(idF, "rb")) == NULL) // deschidere fisier binar
        { printf("Eroare deschidere fisier sursa!"); exit(1);}
    printf("Fisierul %s contine studentii:\n", idF);
    for(i = 1; i <= n; i++)
    {
        fread(&s, sizeof(s), 1, f); //citire din fisier
        printf("%s are nota %d \n", s.nume, s.nota);
    }
    fclose(f);
```

main.c | studenti.in

```
1  Dan
2  Ana
```

input

```
Identificator fisier studenti.in
Numar studenti: 2
Nume student 1: Dan
nota studentului Dan: 10
Nume student 2: Ana
nota studentului Ana: 9
Fisierul studenti.in contine studentii:
Dan are nota 10
Ana are nota 9
```

Cuprinsul cursului de azi

1. Recapitulare. Lucrul cu fișiere
2. Fișiere binare
3. Deschiderea. Funcții de citire/ scriere
4. **Alte funcții**

Funcții de poziționare într-un fișier

1. Modificarea poziției indicatorului de citire/ scriere
2. Poziționarea indicatorului de citire/ scriere la începutul fișierului
3. Testarea sfârșitului de fișier
4. Determinarea poziției curente a indicatorului de citire/ scriere
5. Poziționarea absolută a indicatorului de citire/ scriere

Modificarea poziției indicatorului de citire/ scriere

Funcția **fseek** cu prototipul:

int fseek(FILE* f, long deplasare, int origine);

- ✓ **deplasare** = numărul de octeți cu care se deplasează indicatorul în fișierul **f**
- ✓ **origine** = referința față de care se deplasează indicatorul de citire / scriere
- ✓ parametrul **origine** poate avea valorile:
  - SEEK_SET (=0) poziționare față de începutul fișierului;
  - SEEK_CUR (=1) poziționare față de poziția curentă;
  - SEEK_END (=2) poziționare față de sfârșitul fișierului.

- Funcția returnează valoarea 0 în caz de succes (uneori și în caz de eșec).
- Se semnalează eroare prin returnarea unei valori nenule numai în cazul în care fișierul cu descriptorul **f** nu este deschis.

Modificarea poziției indicatorului de citire/ scriere

Aflarea dimensiunii unui fișier

```c
#include <stdio.h>

void main() {
    FILE *pf = fopen("ceva.txt", "rb");
    if (pf == NULL) {
        printf("Nu gasesc fisierul in dosarul curent!\n");
        return; // Ies din program.
    }
    fseek(pf, 0, SEEK_END);
    long int dim = ftell(pf); // imi spune pozitia curenta in fisier
    printf("Fisierul are %ld octeti.\n", dim);
    // fseek(pf, 0, SEEK_SET); // <-- Daca vreau sa revin la inceput.
    fclose(pf);
}
```

```c
#include <stdio.h>
#include <stdlib.h>

struct numere { int a, b, c;};

int main()
{
    int n;  struct numere num; FILE *fptr;
    if ((fptr = fopen("program.bin","wb")) == NULL)
    {   printf("Error! opening file"); exit(1); }

    for(n = 1; n < 5; ++n) {
        num.a = n; num.b = 2*n; num.c = 3*n - 1;
        fwrite(&num, sizeof(struct numere), 1, fptr);
    }
    fclose(fptr);

    fptr = fopen("program.bin","rb");
    if (fptr == NULL)
    {   printf("Error! opening file"); exit(1); }

    fseek(fptr, -sizeof(struct numere), SEEK_END);
    for(n = 1; n < 5; ++n)  {
        fread(&num, sizeof(struct numere), 1, fptr);
        printf("a = %d\tb = %d\tc = %d\n", num.a, num.b, num.c);
        fseek(fptr, -2*sizeof(struct numere), SEEK_CUR);
    }
    fclose(fptr);

    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>

struct numere { int a, b, c;};

int main()
{
    int n;  struct numere num; FILE *fptr;
    if ((fptr = fopen("program.bin","wb")) == NULL)
    {   printf("Error! opening file"); exit(1); }

    for(n = 1; n < 5; ++n) {
        num.a = n; num.b = 2*n; num.c = 3*n - 1;
        fwrite(&num, sizeof(struct numere), 1, fptr);
    }
    fclose(fptr);

    fptr = fopen("program.bin","rb");
    if (fptr == NULL)
    {   printf("Error! opening file"); exit(1); }

    fseek(fptr, -sizeof(struct numere), SEEK_END);
    for(n = 1; n < 5; ++n)  {
        fread(&num, sizeof(struct numere), 1, fptr);
        printf("a = %d\tb = %d\tc = %d\n", num.a, num.b, num.c);
        fseek(fptr, -2*sizeof(struct numere), SEEK_CUR);
    }
    fclose(fptr);

    return 0;
}
```

main.c program.bin

```
a = 4   b = 8   c = 11
a = 3   b = 6   c = 8
a = 2   b = 4   c = 5
a = 1   b = 2   c = 2
```

```c
#include<stdio.h>
#include<stdlib.h>

typedef struct {
    int nr; char nume[30]; int an; int grupa;
    int n;  int note[15];} Student;

void main()
{
    FILE *f; char s1[20]; Student s;

    printf("\nFisier: "); scanf("%s", s1); f = fopen(s1,"wb");
    if(f == NULL) { printf("Eroare!"); exit(1); }

    printf("Nr.matricol: "); scanf("%d",&s.nr);

    while(!feof(stdin)) // citirea se incheie cu CTRL + D
    {
        printf("Nume: "); getchar(); fgets(s.nume, 20, stdin);
        printf("An: "); scanf("%d", &s.an);
        printf("Grupa: "); scanf("%d", &s.grupa);
        printf("Nr. note (<15): "); scanf("%d", &s.n);
        for(int i = 0; i < s.n; i++)
        {   printf("Nota %d: ", i+1); scanf("%d",&s.note[i]); }
        fwrite(&s, sizeof(Student),  1,f);

        printf("Nr.matricol: "); scanf("%d", &s.nr);
    }

    fclose(f);

    return 0;
}
```

main.c date.bin

```
1  {···Popescu Ana
2  ···············øÌøÑ····ç·······
3  ·······.·······øÌøÑü···Æ=rzb··········ð·······ÀJ·¥Áq···ú··|···Ionascu Daniel
4  ·············øÌøÑ·····························øÌøÑü···Æ=rzb··········ð·······ÀJ·¥Áq···ú··
```

input

```
Fisier: date.bin
Nr.matricol: 123
Nume: Popescu Ana
An: 2
Grupa: 231
Nr. note (<15): 2
Nota 1: 10
Nota 2: 0
Nr.matricol: 124
Nume: Ionascu Daniel
An: 1
Grupa: 143
Nr. note (<15): 3
Nota 1: 6
Nota 2: 9
Nota 3: 0
Nr.matricol:
```

Poziționarea indicatorului de citire/ scriere la începutul fișierului

Funcția **rewind**:

void rewind(FILE *f);

- Funcția are un singur parametru de apel și anume descriptorul fișierului implicat.
- Executarea funcției are ca efect:
  - ✓ poziționarea la începutul fișierului **f** deschis anterior,
  - ✓ resetarea indicatorului de sfârșit de fișier și
  - ✓ resetarea indicatorilor de eroare.
- După apelul lui **rewind** poate urma o operație de scriere sau citire din fișier.

Testarea sfârșitului de fișier

– se realizează prin apelul macro-definiției **feof**:

int feof(FILE* f);

- macro-ul furnizează valoarea indicatorului de sfârșit de fișier asociat lui **f**.
- valoarea indicatorului este stabilită la fiecare operație de citire din fișier.
- valoarea returnată este:
  - ✓ 0, dacă indicatorul are valoarea sfârșit de fișier sau
  - ✓ o valoare diferită de zero, în caz contrar.
- appelul lui ***feof*** trebuie să fie precedat de apelul unei funcții de **citire** din fișier.
- După atingerea sfârșitului de fișier, toate încercările de citire vor eșua, până la apelul funcției **rewind** sau **închiderea și apoi redeschiderea** fișierului.

Determinarea poziției curente a indicatorului de citire/ scriere

– se poate face cu funcția **fgetpos**:

int fgetpos(FILE* f, fpos_t * pozitie);

- după apel, la adresa **pozitie** se află poziția indicatorului de citire/ scriere din fișierul **f,** ca număr relativ al octetului curent
- primul octet are numărul 0
- valoarea returnată poate fi folosită pentru poziționare cu funcția **fsetpos**
- funcția returnează:
  - ✓ valoarea 0, în caz de succes ,
  - ✓ o valoare nenulă, în caz de eroare.

Determinarea poziției curente a indicatorului de citire/ scriere

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    char idf[] = "ceva.txt";
    FILE *f = fopen(idf, "w+");
    fpos_t poz;

    fputs("Acesta este un text", f);
    fgetpos(f, &poz);

    fsetpos(f, &poz );
    fputs("Acesta este un text nou", f);

    fclose(f);

return 0;
}
```

main.c | ceva.txt

```
1  Acesta este un textAcesta este un text nou
```

Determinarea poziției curente a indicatorului de citire/ scriere

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    char idf[] = "ceva.txt";
    FILE *f = fopen(idf, "w+");
    fpos_t poz;
    fgetpos(f, &poz);
    fputs("Acesta este un text", f);

    fsetpos(f, &poz );
    fputs("Acesta este un text nou", f);

    fclose(f);

return 0;
}
```

- Suprascrie

main.c | ceva.txt

```
1  Acesta este un text nou
```

Determinarea poziției curente a indicatorului de citire/ scriere

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    char idf[] = "ceva.txt";
    FILE *f = fopen(idf, "w+");
    fpos_t poz;
    fgetpos(f, &poz);
    fputs("Acesta este un text foarte foarte lung", f);


    fsetpos(f, &poz );
    fputs("Acesta este un text nou", f);

    fclose(f);

    return 0;
}
```

- Suprascrie

main.c | ceva.txt

```
1  Acesta este un text nourte foarte lung
```

Determinarea poziției curente a indicatorului de citire/ scriere

– se poate face cu funcția **ftell**:

long int ftell(FILE* f);

- după apel, la adresa poziție se află poziția indicatorului de citire
- funcția returnează:
  - ✓ poziția în fișierul f a indicatorului de citire/ scriere în caz de succes
  - ✓ -1L în caz contrar.
- dacă fișierul este binar, poziția este dată în număr de octeți față de începutul fișierului (dimensiunea maximă a unui fișier în C este de 2^{31}-1 octeți ~ 2GB)
- valoarea poate fi folosită pentru poziționare cu funcția **fseek**.

Poziționarea absolută a indicatorului de citire/ scriere

– se face cu funcția **fsetpos**:

int fsetpos(FILE* f, const fpos_t pozitie);

- Indicatorul de citire/ scriere se mută în fișierul cu descriptorul **f** la octetul cu numărul indicat de parametrul **pozitie** (care poate fi o valoare obținută prin apelul lui **fgetpos**).
- Ambele funcții (**fgetpos** și **fsetpos**) resetează indicatorul de sfârșit de fișier.

Alte funcții pentru lucrul cu fișiere

- *Redenumirea sau mutarea unui fișier*
- *Ștergerea unui fișier*
- *Furnizare unui nume de fișier*

Redenumirea sau mutarea unui fișier existent

– se realizează prin apelul funcției **rename** cu prototipul:

int rename(const char* n_vechi, const char* n_nou);

- unde **n_vechi** este vechiul nume al fișierului, iar **n_nou**=numele nou.
- dacă numele vechi conține numele discului (de exemplu C:), numele nou trebuie să conțină același nume de disc.
- dacă numele vechi conține o cale, numele nou **nu** este obligat să conțină aceeași cale. ***Folosind o altă cale se obține mutarea fișierului pe disc.***
- **folosind aceeași cale (sau nefolosind calea) se obține redenumirea fișierului.**
- **nu** sunt permise caracterele (?, *) în cele două nume.
- funcția returnează valoarea:
  - ✓ 0, în caz de succes
  - ✓ -1, în caz de eroare.

Redenumirea sau mutarea unui fișier existent

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  int main(){
6      char sir[81];
7      FILE *f = fopen("ceva.txt", "w");
8      printf( "Introdu 4 cuvinte: \n" );
9      for( int i = 0; i < 4; i++ ) {
10         scanf( "%s", sir);
11         fprintf(f, "%s ", sir);
12         fflush(stdin);
13     }
14     fclose(f);
15
16     if (!rename("ceva.txt", "altceva.in")) {
17         FILE *g = fopen("altceva.in","r");
18         if (g == NULL) {printf("Eroare");exit(1);}
19         fgets( sir, sizeof(sir), g );
20         printf( "%d octeti are sirul citit din fisier: \"%s\"", strlen(sir), sir );
21         fclose(g);
22     }
23     else printf("Nu s-a putut redenumi fisierul.");
24     return 0;
25 }
```

main.c altceva.in

```
1  Ana are doua mere
```

```
Introdu 4 cuvinte:
Ana
are
doua
mere
18 octeti are sirul citit din fisier: "Ana are doua mere "
```

Ștergerea unui fișier existent

- se poate realiza prin apelul funcției **remove**, care are următorul prototip:

int remove(const char* identificator);

- parametrul identificator reprezintă numele extern al fișierului și care poate să conțină calea de căutare a fișierului.

Ștergerea unui fișier existent

main.c

```c
int main(){
    char sir[81], idf[] = "ceva.txt";
    FILE *f = fopen(idf, "w");
    printf( "Introdu 4 cuvinte: \n" );
    for( int i = 0; i < 4; i++ ) {
        scanf( "%s", sir);
        fprintf(f, "%s ", sir);
        fflush(stdin);
    }
    fclose(f);

    f = fopen("ceva.txt","r");
    if (f == NULL) {printf("Eroare");exit(1);}
    fgets( sir, sizeof(sir), f );
    printf( "Sirul citit din fisierul %s: \n\"%s\"", idf, sir );
    fclose(f);
    remove("ceva.txt");
```

input

```
Introdu 4 cuvinte:
Tema
cateva
EXERCITII
usoare
Sirul citit din fisierul ceva.txt:
"Tema cateva EXERCITII usoare "
```

Furnizare unui nume de fișier

Funcția

char *tmpnam(char* nume_fisier)

- furnizează un nume de fisier pe care îl pune în nume_fisier care nu există în directorul curent

Furnizare unui nume de fișier

```c
int main() {    char sir[81], *idf;
    if ((idf = tmpnam(NULL)) != NULL) {
        printf("%s poate fi utilizat ca nume de fisier.\n", idf);
        FILE *f = fopen(idf, "w");
        printf( "Introdu 4 cuvinte: \n" );
        for( int i = 0; i < 4; i++ ) {
            scanf( "%s", sir);
            fprintf(f, "%s ", sir);
            fflush(stdin);
        }
        fclose(f);
        f = fopen(idf, "r");
        fgets (sir, sizeof(sir), f);
        printf("%s", sir);
        fclose(f);
    }
    else printf("Nu se poate crea un nume de fisier.\n");
    return 0;
```

input

```
main.c:(.text+0xe): warning: the use of `tmpnam' is dangerous, better use
 `mkstemp'
/tmp/file5665Mt poate fi utilizat ca nume de fisier.
Introdu 4 cuvinte:
Am
luat
note
mari
Am luat note mari
```

BUFFERED STREAMS

- Operațiile cu HDD sunt mult mai încete decât cele cu memoria (RAM) => se folosește un buffer pentru a citi/ scrie blocuri mai mari în memorie înainte de a reciti/rescrie efectiv buffer-ul pe HDD
- Toate stream-urile deschise cu **fopen()** folosesc buffere dacă se știe că nu se referă la un dispozitiv interactiv
- Tipuri de buffering: *full buffer, line buffer, no buffer*
- Bufferul se poate schimba cu:

void setbuf (FILE * stream, char * buffer);

- Tipul de buffering se poate schimba cu:

int setvbuf (FILE * stream, char * buffer, int mode, size_t size);

Golirea explicită a zonei tampon a unui fișier

- se realizează prin apelul funcției **fflush** cu următorul prototip:

int fflush(FILE* f);

- dacă fișierul cu descriptorul **f** are asociat un buffer de ieșire, funcția scrie în fișier toate informațiile din acesta, la poziția curentă
- dacă fișierul are asociat un buffer de intrare, funcția îl golește. În caz de succes returnează valoarea zero, iar în caz de eroare valoarea constantei simbolice EOF (definită în stdio.h)
- înainte de a citi un șir de caractere de la tastatură, **buffer-ul trebuie golit** pentru a preveni citirea unui șir vid (datorită unei perechi CR/LF rămase în buffer de la o citire anterioară a unei valori numerice). Ștergerea se realizează prin apelul:

fflush(stdin);

Golirea explicită a zonei tampon a unui fișier

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>


int main() {
    char sir[81], idf[] = "ceva.txt";
    FILE *f = fopen(idf, "w");
    printf( "Introdu 4 cuvinte: \n" );
    for( int i = 0; i < 4; i++ ) {
        scanf( "%s", sir);
        fprintf(f, "%s ", sir);
        fflush(stdin);
    }
    sleep(10);// asteapta 10 secunde
    fflush(f);
    fclose(f);

    f = fopen(idf, "r");
    fgets (sir, sizeof(sir), f);
    printf("%s", sir);
    fclose(f);

return 0;
}
```

int fflush (FILE * stream);

Pentru a forța scrierea buffer-ului stream-ului în fișier

Cursul 9

Fișiere binare. Funcții specifice de manipulare

Cursul 10

Structuri de date complexe și autoreferite